

Baseline Notebook

Setup Environment

```
In [ ]: # DO NOT MODIFY THE CODE IN THIS CELL
!pip install -q utstd

from utstd.folders import *
from utstd.ipyrenders import *

at = AtFolder(
    course_code=36106,
    assignment="AT3",
)
at.run()

import warnings
warnings.simplefilter(action='ignore')
```

Student Information

```
In [ ]: group_name = "36106-26AU-AT3-Group 24"
student_name = "Nana Ama Goldwater"
student_id = "26137455"
```

```
In [ ]: # Do not modify this code
print_tile(size="h1", key='group_name', value=group_name)
```

```
In [ ]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_name', value=student_name)
```

```
In [ ]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h1", key='student_id', value=student_id)
```

0. Python Packages

0.a Install Additional Packages

If you are using additional packages, you need to install them here using the command: !

```
pip install <package_name>
```

```
In [ ]: # No additional packages required. scikit-learn, pandas, numpy and altair are already available i
```

0.b Import Packages

```
In [ ]: import numpy as np
import pandas as pd
import altair as alt

from sklearn.dummy import DummyClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score,
    confusion_matrix, classification_report,
)

# Display options
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 200)
```

```
RANDOM_STATE = 42
```

A. Assess Baseline Model

```
In [ ]: # DO NOT MODIFY THE CODE IN THIS CELL
# Load data
try:
    X_train = pd.read_csv(at.folder_path / 'X_train.csv')
    y_train = pd.read_csv(at.folder_path / 'y_train.csv')

    X_val = pd.read_csv(at.folder_path / 'X_val.csv')
    y_val = pd.read_csv(at.folder_path / 'y_val.csv')

    X_test = pd.read_csv(at.folder_path / 'X_test.csv')
    y_test = pd.read_csv(at.folder_path / 'y_test.csv')
except Exception as e:
    print(e)
```

A.1 Generate Predictions with Baseline Model

```
In [ ]: import os
import numpy as np
import pandas as pd
from sklearn.dummy import DummyClassifier

RANDOM_STATE = 42
DATA_DIR = at.folder_path

# — Option A: variables already in memory from earlier cells —————
# If X_train / y_train etc. exist in the session, use them directly and optionally save for persistence
splits_in_memory = all(v in globals() for v in
                        ['X_train', 'X_val', 'X_test', 'y_train', 'y_val', 'y_test'])

if splits_in_memory:
    print("Splits found in memory → using directly from memory.")
    X_train = globals()['X_train']
    X_val = globals()['X_val']
    X_test = globals()['X_test']
    y_train = globals()['y_train']
    y_val = globals()['y_val']
    y_test = globals()['y_test']

    # Attempt to save the splits to CSV for future use, but handle OSError gracefully
    try:
        print(" Attempting to save splits to CSV for persistence...")
        os.makedirs(str(DATA_DIR), exist_ok=True) # Ensure directory exists
        for name, obj in [('X_train', X_train), ('X_val', X_val), ('X_test', X_test),
                          ('y_train', y_train), ('y_val', y_val), ('y_test', y_test)]:
            obj.to_csv(DATA_DIR / f'{name}.csv', index=False)
        print(" Saved successfully.")
    except OSError as e:
        print(f" Warning: Failed to save splits to CSV due to Google Drive issue: {e}. Proceeding...")

# — Option B: load from Google Drive if not in memory —————
else: # splits_in_memory is False, so try to load from disk
    if not os.path.exists(DATA_DIR / 'X_train.csv'):
        print("Files not found. Choose one of the options below:\n")
        print(" OPTION 1 – Mount Google Drive:")
        print("   from google.colab import drive")
        print("   drive.mount('/content/drive/')")
        print("   DATA_DIR = '/content/drive/MyDrive/<your-folder>/' # ← update path\n")
        print(" OPTION 2 – Upload files directly from your machine:")
        print("   from google.colab import files")
        print("   files.upload() # select all 6 CSVs at once\n")
        print(" OPTION 3 – Re-run all earlier pipeline cells to rebuild splits in memory,")
        print("           then re-run this cell.\n")
        raise FileNotFoundError(
            "CSV files missing. Follow one of the options printed above, "
            "update DATA_DIR if needed, then re-run this cell."
        )
    else:
        print("Splits not found in memory → loading from CSV...")
```

```

# — Load —————
X_train = pd.read_csv(DATA_DIR / 'X_train.csv')
X_val   = pd.read_csv(DATA_DIR / 'X_val.csv')
X_test  = pd.read_csv(DATA_DIR / 'X_test.csv')

y_train = pd.read_csv(DATA_DIR / 'y_train.csv')
y_val   = pd.read_csv(DATA_DIR / 'y_val.csv')
y_test  = pd.read_csv(DATA_DIR / 'y_test.csv')

# — Squeeze targets to 1-D —————
y_train_s = y_train.squeeze('columns') if y_train.shape[1] == 1 else y_train.iloc[:, -1]
y_val_s   = y_val.squeeze('columns')   if y_val.shape[1]   == 1 else y_val.iloc[:, -1]
y_test_s  = y_test.squeeze('columns')  if y_test.shape[1]  == 1 else y_test.iloc[:, -1]

print("Dataset shapes")
print(f" X_train: {X_train.shape}   y_train: {y_train_s.shape}")
print(f" X_val  : {X_val.shape}     y_val  : {y_val_s.shape}")
print(f" X_test : {X_test.shape}    y_test : {y_test_s.shape}")

print("\nTarget class distribution (proportion of positive class = 1)")
print(f" train: {y_train_s.mean():.4f}   ({int(y_train_s.sum())}/{len(y_train_s)})")
print(f" val  : {y_val_s.mean():.4f}   ({int(y_val_s.sum())}/{len(y_val_s)})")
print(f" test : {y_test_s.mean():.4f}   ({int(y_test_s.sum())}/{len(y_test_s)})")

# — Baseline models —————
baseline_majority = DummyClassifier(strategy='most_frequent')
baseline_stratified = DummyClassifier(strategy='stratified', random_state=RANDOM_STATE)

baseline_majority.fit(X_train, y_train_s)
baseline_stratified.fit(X_train, y_train_s)

preds = {
    'majority' : {s: baseline_majority.predict(globals()[f'X_{s}']) for s in ['train', 'val', 'test']},
    'stratified' : {s: baseline_stratified.predict(globals()[f'X_{s}']) for s in ['train', 'val', 'test']}
}
probas = {
    'majority' : {s: baseline_majority.predict_proba(globals()[f'X_{s}'])[:, 1] for s in ['train', 'val', 'test']},
    'stratified' : {s: baseline_stratified.predict_proba(globals()[f'X_{s}'])[:, 1] for s in ['train', 'val', 'test']}
}

print("\nBaseline models fitted. Sample predictions on the first 10 validation rows:")
print(" majority   :", preds['majority']['val'][:10])
print(" stratified :", preds['stratified']['val'][:10])
print(" actual      :", y_val_s.iloc[:10].to_numpy())

```

A.2 Selection of Performance Metrics

Provide some explanations on why you believe the performance metrics you chose is appropriate

```

In [ ]: # Confirm class imbalance numerically – this is the main driver of metric choice.
balance_df = pd.DataFrame({
    'split' : ['train', 'val', 'test'],
    'n'      : [len(y_train_s), len(y_val_s), len(y_test_s)],
    'n_pos'  : [int(y_train_s.sum()), int(y_val_s.sum()), int(y_test_s.sum())],
    'pct_pos' : [y_train_s.mean(), y_val_s.mean(), y_test_s.mean()],
})
balance_df['pct_neg'] = 1 - balance_df['pct_pos']
display(balance_df.style.format({'pct_pos': '{:.2%}', 'pct_neg': '{:.2%}')))

# Visualise class balance per split
long_df = balance_df.melt(
    id_vars='split',
    value_vars=['pct_pos', 'pct_neg'],
    var_name='class', value_name='proportion'
).replace({'pct_pos': 'positive (1)', 'pct_neg': 'negative (0)'})

alt.Chart(long_df).mark_bar().encode(
    x=alt.X('split:N', title='Split'),
    y=alt.Y('proportion:Q', title='Proportion', axis=alt.Axis(format='%')),
    color=alt.Color('class:N', title='Class'),
    tooltip=['split', 'class', alt.Tooltip('proportion:Q', format='.2%')]
)

```

```
).properties(width=450, height=250, title='Class balance per split')
```

```
In [ ]: performance_metrics_explanations = """
Use case: repeat-purchase classification predict, for each customer, whether they
will place at least one order in the prediction window after the cutoff date.
Target is binary (1 = customer reorders, 0 = does not).

Class balance is the deciding factor for metric choice. Retail repeat-purchase data
is typically imbalanced (most customers don't reorder in any given short window), so
accuracy alone is misleading a model that always predicts 0 can score >80% accuracy
while being commercially useless. The chosen metrics below address that.

Primary metrics
- F1 score (positive class). Harmonic mean of precision and recall on the class
  we actually care about (customers who will reorder). It is sensitive to both
  false positives (wasted marketing spend) and false negatives (missed customers).
- ROC-AUC. Threshold-independent; tells us how well the model RANKS reorders
  above non-reorders. Useful when downstream teams will choose their own cutoff
  (e.g. 'target the top-decile by score').
- Average Precision (PR-AUC). On imbalanced data, PR-AUC is more informative
  than ROC-AUC because it focuses on the minority (positive) class. A random
  model's PR-AUC equals the positive rate, not 0.5 so improvements are
  interpretable in business terms.

Supporting metrics
- Precision (positive class). What fraction of customers we flagged as 'likely
  to reorder' actually did? Drives campaign efficiency / cost-per-conversion.
- Recall (positive class). What fraction of actual reorderers did we catch?
  Drives coverage / opportunity cost.
- Confusion matrix. Reported on the validation set so the four cells
  (TP / FP / TN / FN) can be tied directly to business cost.
- Accuracy. Reported for completeness only, it is NOT used to compare models.

Reading the baselines
- DummyClassifier(strategy='most_frequent') always predicts the majority class
  (the 'do nothing' strategy). Its F1 on the positive class is 0 and its AUC
  is 0.5 by construction. It establishes the trivial accuracy floor.
- DummyClassifier(strategy='stratified') predicts randomly from the class prior.
  Its F1  $\approx$  positive-class rate and its AUC = 0.5. It establishes the trivial
  random-ranking floor.

Any candidate model in the classification notebook MUST beat both baselines on
F1 (positive class) AND on PR-AUC to be considered useful.
"""
```

```
In [ ]: # DO NOT MODIFY THE CODE IN THIS CELL
print_tile(size="h3", key='performance_metrics_explanations', value=performance_metrics_explanations)
```

```
In [ ]: ethics_metrics_note = """
Ethics note – why metric choice matters for fairness

Accuracy is rejected as a primary metric not only because it is dominated
by the majority class, but also for an ethics reason: an accuracy-
optimising model is incentivised to ignore the minority (positive) class
entirely. In a retention context that means systematically de-prioritising
customers who would have benefited from outreach likely overlapping
with under-engaged, regional, or otherwise marginal segments.

F1 and PR-AUC put the minority class at the centre of the optimisation,
which is the metric-level safeguard against silent under-service.
"""
print_tile(size="h3", key='ethics_metrics_note', value=ethics_metrics_note)
```

A.3 Baseline Model Performance

Provide some explanations on model performance

```
In [ ]: # Scoring helper
def score_split(y_true, y_pred, y_proba):
    return {
        'accuracy' : accuracy_score(y_true, y_pred),
        'precision' : precision_score(y_true, y_pred, zero_division=0),
        'recall' : recall_score(y_true, y_pred, zero_division=0),
        'f1' : f1_score(y_true, y_pred, zero_division=0),
        'roc_auc' : roc_auc_score(y_true, y_proba) if len(np.unique(y_true)) > 1 else np.nan,
```

```

        'pr_auc' : average_precision_score(y_true, y_proba) if len(np.unique(y_true)) > 1 else
    }

y_by_split = {'train': y_train_s, 'val': y_val_s, 'test': y_test_s}
rows = []
for model_name in ['majority', 'stratified']:
    for split in ['train', 'val', 'test']:
        m = score_split(y_by_split[split], preds[model_name][split], probas[model_name][split])
        m['model'] = model_name
        m['split'] = split
        rows.append(m)

results = pd.DataFrame(rows[
    ['model', 'split', 'accuracy', 'precision', 'recall', 'f1', 'roc_auc', 'pr_auc']
])
print("Baseline performance – all splits, both strategies:")
display(
    results.style.format({c: '{:.4f}' for c in
        ['accuracy', 'precision', 'recall', 'f1', 'roc_auc', 'pr_auc']})
)

# Confusion matrix on validation (majority strategy)
cm = confusion_matrix(y_val_s, preds['majority']['val'])
print("\nConfusion matrix – majority baseline on validation set:")
print(pd.DataFrame(
    cm,
    index=['actual_0', 'actual_1'],
    columns=['pred_0', 'pred_1']
))

print("\nClassification report – stratified baseline on validation set:")
print(classification_report(y_val_s, preds['stratified']['val'], zero_division=0))

# Visualise metric comparison on validation
val_long = (
    results.query("split == 'val'")
    .melt(id_vars=['model', 'split'],
        value_vars=['accuracy', 'precision', 'recall', 'f1', 'roc_auc', 'pr_auc'],
        var_name='metric', value_name='score')
)
alt.Chart(val_long).mark_bar().encode(
    x=alt.X('model:N', title=None, axis=alt.Axis(labels=False, ticks=False)),
    y=alt.Y('score:Q', title='Score', scale=alt.Scale(domain=[0, 1])),
    color=alt.Color('model:N', title='Baseline'),
    column=alt.Column('metric:N', title='Metric',
        sort=['accuracy', 'precision', 'recall', 'f1', 'roc_auc', 'pr_auc']),
    tooltip=['model', 'metric', alt.Tooltip('score:Q', format='%.4f')]
).properties(width=80, height=220, title='Baseline metrics on the validation set')

```

```

In [ ]: baseline_performance_explanations = """
Headline numbers (validation set)
- majority : accuracy ≈ proportion of the majority (negative) class; precision,
            recall, F1 on the positive class are all 0 by construction; ROC-AUC
            and PR-AUC are at their trivial floors.
- stratified : accuracy drops toward the no-skill level ( $p^2 + (1-p)^2$  where  $p$  is the
            positive rate); F1 ≈ positive rate; ROC-AUC ≈ 0.5; PR-AUC ≈ positive rate.
- Train vs val vs test scores are essentially identical for both baselines, expected, because r
            train and val for a real model is therefore attributable to that model, not
            to dataset noise.

```

Interpretation

- The majority baseline highlights why accuracy is the wrong primary metric: it scores high on accuracy while being commercially useless (it never flags a single customer as 'will reorder'). Any model that beats it on accuracy but not on F1 or PR-AUC has learned nothing useful.
- The stratified baseline provides a non-zero F1 floor that ANY proposed model must clear to demonstrate genuine signal. It also gives a PR-AUC equal to the positive-class rate, which is the correct 'no information' reference for PR-AUC on imbalanced data.
- The confusion matrix for the majority baseline contains only TN and FN cells. From a business standpoint this means we lose 100% of the reorder opportunity (recall = 0). This is the cost of the 'do nothing' policy and sets the business floor any deployed model must improve on.

Acceptance criteria for the classification notebook

- F1 on positive class > stratified baseline F1 by a clear margin on validation.

```
- PR-AUC > positive-class rate (i.e. better than random ranking).  
- ROC-AUC meaningfully above 0.5 (target  $\geq 0.65$  as a first milestone).  
- Test-set performance within ~1–2 percentage points of validation, larger  
  gaps suggest validation leakage or overfit hyperparameters.  
- Confusion-matrix-derived precision and recall must be interpretable in  
  business terms (campaign efficiency vs coverage).  
"""
```

```
In [ ]: # DO NOT MODIFY THE CODE IN THIS CELL  
print_tile(size="h3", key='baseline_performance_explanations', value=baseline_performance_explanations)
```